TyC Programming Language Specification

Version 1.0 - January 2026

Table of Contents

1. Introduction
2. Program Structure
3. Lexical Structure
4. Type System
5. Expressions
6. Statements
7. Type Inference
8. Scope Rules
9. Input and Output
10. Example Programs

Introduction

TyC, pronounced "type-see", is a simple C-like programming language designed primarily for students practicing implementing a simple compiler. TyC features a complete type inference system with strict type checking where each operator has well-defined type requirements, making type checking straightforward while maintaining C-like syntax.

Despite its simplicity, TyC includes most important features of a procedural programming language such as functions, variables, control flow structures (if, while, for, switch-case), and complete type inference.

Program Structure

As its simplicity, a TyC compiler does not support compiling many files, so a TyC program is written in just one file only. A TyC program consists of a sequence of struct declarations and function declarations.

The entry point of a TyC program is a function named `main` that takes no parameters and returns `void`. Each such function in a TyC program is an entry point of the program.

Function Declaration

A function declaration has the following form:

```
<return_type> <identifier>(<parameter_list>) {
    <statement_list>
}
```

Where:

- `<return_type>` is a type which is described in Section 4, or can be omitted for inferred return type
- `<identifier>` is the function name
- `<parameter_list>` is a comma-separated list of parameter declarations (or empty)
- `<statement_list>` is a sequence of statements

A parameter declaration has the form:

```
<type> <identifier>
```

where `<identifier>` is the parameter name and `<type>` must be an explicit type (`int`, `float`, `string`, or a struct type name). **Note:** Parameters cannot use `auto` for type inference - the type must be explicitly declared.

The `<statement_list>` will be described in Statements.

Function Overloading: TyC does not support function overloading. In a TyC program, function names must be unique - there cannot be two functions with the same name, regardless of their parameter types or return types. This restriction simplifies type inference, as the type of a function call can be determined solely by the function name without needing to consider multiple function signatures with different parameter types.

Example

```
int add(int x, int y) {
    return x + y;
}

void main() {
    auto result = add(3, 5);
    printInt(result);
}
```

Example with Inferred Return Type

When the return type is omitted, it is inferred from the return statements in the function:

```
// Return type inferred as int
add(int x, int y) {
    return x + y;
}

// Return type inferred as float
multiply(float a, float b) {
    return a * b;
}

// Return type inferred as void (no return statement)
greet(string name) {
    printString("Hello, ");
```

```
    printString(name);
}

void main() {
    auto sum = add(3, 5);              // sum: int
    auto product = multiply(2.5, 3.0);  // product: float
    greet("World");
}
```

Lexical Structure

Character Set

The character set of TyC is extended ASCII (characters with code points 0-255). Blank (`' '`), tab (`'\t'`), form feed (i.e., the ASCII FF) (`'\f'`), carriage return (i.e., the ASCII CR – `'\r'`) and newline (i.e., the ASCII LF – `'\n'`) are whitespace characters. The `'\n'` is used as newline character in TyC.

This definition of lines can be used to determine the line numbers produced by a TyC compiler.

Comment

There are two types of comment in TyC: block and line. A block comment starts with `"/*"` and ignores all characters (except EOF) until it reaches `"*/"`. A line comment ignores all characters from `"//"` to the end of the current line, i.e., when reaching end of line or end of file.

For example:

```
/* This is a block comment, that
may span in many lines*/
auto x = 5; //this is a line comment
```

The following rules are enforced in TyC:

- Comments are not nested
- `"/*"` and `"*/"` have no special meaning in any line comment
- `"//"` has no special meaning in any block comment

For example:

```
/* This is a block comment so // has no meaning here */
//This is a line comment so /* has no meaning here
```

Identifier

Identifiers are used to name variables, functions, and parameters.
Identifiers begin with a letter (A-Z or a-z) or underscore (_), and may contain letters, underscores, and digits

(0-9). TyC is case-sensitive, therefore the following identifiers are distinct: `MyVar`, `myvar`, and `MYVAR`.

Keyword

The following character sequences are reserved as keywords and cannot be used as identifiers:

auto	**break**	**case**	**continue**	**default**	**else**
float	**for**	**if**	**int**	**return**	**string**
---	---	---	---	---	---
struct	**switch**	**void**	**while**		

Operator

The following is a list of **valid** operators along with their meaning. Note the applicable types for each operator:

Operator	Meaning	Applicable Type
`+`	Addition or unary plus	int or float
`-`	Subtraction or unary minus	int or float
`*`	Multiplication	int or float
`/`	Division	int or float
`%`	Modulus	int only
`==`	Equal	int or float
`!=`	Not equal	int or float
`<`	Less than	int or float
`>`	Greater than	int or float
`<=`	Less than or equal	int or float
`>=`	Greater than or equal	int or float
`||`	Logical OR	int only
`&&`	Logical AND	int only
`!`	Logical NOT	int only
`++`	Increment	int only
`--`	Decrement	int only
`=`	Assignment	any
`.`	Member access	struct

Separator

The following characters are the **separators**: left brace (`{`), right brace (`}`), left parenthesis (`(`), right parenthesis (`)`), semicolon (`;`), comma (`,`), and colon (`:`).

Literal

A literal is a source representation of a value of an integer, floating-point, or string type.

Integer literal

Integer literals are values that **are always expressed in decimal** (base 10). A decimal number is a string of digits (0-9) and is at least one digit long. An integer literal may be preceded by a minus sign (`-`) to indicate a negative value.
The following are valid integer numbers: `0 100 255 2500 -45`
Integer literals are of type **int**.

Float literal

Float literals are values that represent floating-point numbers. A float literal consists of digits and may include a decimal point and/or an exponent part.

A float literal must have at least one digit. If a decimal point is present, there must be at least one digit either before or after the decimal point (or both). The decimal point may be omitted if an exponent part is present.

An exponent part consists of the letter `e` or `E`, optionally followed by a plus sign `+` or minus sign `-`, followed by one or more digits. The exponent sign is optional; if omitted, the exponent is positive. The exponent part itself is optional when a decimal point is present, but required when there is no decimal point.

The following are valid float literals: `0.0 3.14 1.23e4 5.67E-2 1. .5 1e4 2E-3`
Float literals are of type **float**.

String literals

String literals consist of zero or more characters enclosed by double quotes (`"`). A string literal can contain any character from the extended ASCII character set (code points 0-255), with the following restrictions:

- **Newline (`\n`) and carriage return (`\r`) characters** cannot appear directly in a string literal. They must be represented using escape sequences (`\n` and `\r` respectively).
- **Backslash (`\`)** and **double quote (`"`)** characters must be escaped when they appear in the string content (using `\\` and `\"` respectively).
- **All other characters**, including unprintable ASCII characters (0-31) other than `\n` and `\r`, can appear directly in string literals without requiring escape sequences. However, for readability and portability, it is recommended to use escape sequences for unprintable characters when possible.

Escape sequences are used to represent special characters within a string. All the supported escape sequences are as follows:

```
\b   backspace (ASCII 8)
\f   formfeed (ASCII 12)
\r   carriage return (ASCII 13)
```

```
\n   newline (ASCII 10)
\t   horizontal tab (ASCII 9)
\"   double quote (ASCII 34)
\\   backslash (ASCII 92)
```

Only these escape sequences are valid. Hex escapes such as `\x01` or `\x80` are not supported and will cause an `ILLEGAL_ESCAPE` error; unprintable and extended ASCII characters may be written directly in the source when the editor allows.

String Token Processing:

- When a valid string literal is recognized, the lexer automatically removes (strips) the enclosing double quotes from both ends. The token value contains only the string content without the quotes.
- For error cases (`ILLEGAL_ESCAPE` and `UNCLOSE_STRING`), the lexer removes the opening double quote, but the error message includes the problematic content.
- For `ILLEGAL_ESCAPE` errors, the error message includes the string content from the beginning (without the opening quote) up to and including the illegal escape sequence (i.e., the backslash and the character that follows it that makes it illegal).

Error Detection Order: The lexer checks for errors in the following order (first match wins):

1. **Illegal escape sequences** are detected first. An illegal escape is any backslash followed by a character that is not one of the supported escape characters (`b`, `f`, `r`, `n`, `t`, `"`, `\`), and is not followed by a newline or carriage return.
2. **Unclosed strings** are detected if the string literal is not closed before encountering a newline, carriage return, or end of file.
3. If neither error occurs, a **valid string literal** is recognized.

It is a compile-time error for:

- A newline (`\n`) or carriage return (`\r`) character to appear directly (unescaped) inside a string literal.
- An EOF character to appear inside a string literal (i.e., the string literal is not closed before end of file).
- An illegal escape sequence to appear (any backslash followed by a character that is not one of the supported escape characters: `b`, `f`, `r`, `n`, `t`, `"`, `\`).

The following are valid examples of string literals:

```
"This is a string containing tab \t"
"He asked me: \"Where is John?\""
""
```

A string literal has a type of **string**.

Type System

Types limit the values that a variable can hold (e.g., an identifier x whose type is int cannot hold value "hello"...), the values that an expression can produce, and the operations supported on those values (e.g., we

can not apply operation + in two string values...).

Primitive Type

Integer

The keyword `int` is used to represent an integer type. A value of type integer may be positive or negative. Only these operators can act on integer values:
`+ - * / % == != < <= > >= && || ! ++ --`

Float

The keyword `float` represents a floating-point type. A value of type float may be positive or negative. Only these operators can act on float values:
`+ - * / == != < <= > >=`

Note that the modulus operator (`%`) and logical operators (`&&`, `||`, `!`) do not apply to float values. Also, increment (`++`) and decrement (`--`) operators do not apply to float values.

String

The keyword `string` expresses the string type. There are no operators that act on string values, except for assignment and function parameters/return values. The string type is used primarily for input/output operations.

Void

The keyword `void` is used to express the void type. This type is only used as a return type of a function when it has nothing to return. This type is **not** allowed to use for a variable or parameter declaration.

Struct Type

The keyword `struct` is used to define a composite data type that groups together variables of different types. A struct declaration defines a new type that can be used throughout the program.

Struct Declaration

A struct declaration has the following form:

```
struct <identifier> {
    <type1> <member1>;
    <type2> <member2>;
    ...
    <typeN> <memberN>;
};
```

Where:

- `<identifier>` is the struct name

- Each `<type>` must be an explicit type (`int`, `float`, `string`, or another struct type)
- Each `<member>` is a member name (identifier)
- A struct can have zero or more members.

Important Rules:

- Struct members must have explicit types - they cannot use `auto` for type inference
- Nested struct definitions are not supported: Struct declarations cannot be nested (you cannot declare a struct inside another struct declaration)
- Struct members can be struct types: However, struct members can be of other struct types (using previously declared struct types)
- Struct names must be unique in the program
- Struct members can be of primitive types (`int`, `float`, `string`) or other struct types (that are declared before use)

For example:

```
struct Empty {};  // Valid: empty struct with no members

struct Point {
    int x;
    int y;
};

struct Person {
    string name;
    int age;
    float height;
};
```

Struct Variable Declaration

A struct variable can be declared using the struct name as the type:

```
<struct_name> <identifier>;                    // without initialization
<struct_name> <identifier> = {<member_list>};  // with initialization
```

Where:

- `<struct_name>` is the name of a previously declared struct type
- `<identifier>` is the variable name
- `<member_list>` is a comma-separated list of initialization expressions (or empty for an empty struct)

Struct Initialization Rules:

- When initializing a struct with `{<member_list>}`, the number of expressions in `<member_list>` must match the number of members in the struct

- Each expression in `<member_list>` must be in the same order as the struct members
- The type of each initialization expression must match the type of the corresponding struct member:
  - The first expression initializes the first member
  - The second expression initializes the second member
  - And so on...
- Each expression in `<member_list>` can be a literal, a variable, a function call, a struct literal (for struct members that are struct types), or any other expression that evaluates to the correct type
- **Initialization of struct members that are struct types:** If a struct member is of another struct type, it can be initialized using a struct literal (e.g., `Point3D p = {{1, 2}, 3};`) or a variable of that struct type (e.g., `Point2D p2 = {1, 2}; Point3D p3 = {p2, 3};`)
- **Struct literals in function calls:** Struct literals can be used as function arguments (e.g., `f({4, 5})`), where the struct literal's type is determined by the function parameter type
- If a struct variable is declared without initialization, all its members have undefined values until assigned

For example:

```
Point p1;                       // uninitialized
Point p2 = {10, 20};           // initialized: x=10, y=20

Person person1;                // uninitialized
Person person2 = {"John", 25, 1.75};  // initialized: name="John", age=25,
height=1.75
```

Struct Member Access

Struct members are accessed using the dot (`.`) operator. The left-hand side may be any expression that evaluates to a struct type (e.g., a variable, a function call that returns a struct, or a parenthesized expression):

```
<expr>.<member_name>
```

where `<expr>` must have a struct type and `<member_name>` must be a member of that struct.

For example:

```
Point p = {10, 20};
p.x = 30;           // assign to member x
auto x_coord = p.x; // read member x
printInt(p.x);      // use member x in expression
p.x++;              // increment member x (parsed as (p.x)++)
```

Struct Operations

- **Assignment**: Struct values can be assigned using `=`. Assignment copies all member values.
- **Equality**: Struct types do not support equality operators (`==`, `!=`).

- **Arithmetic**: Struct types do not support arithmetic operators.
- **Member Access**: The dot (`.`) operator is used to access struct members.

For example:

```
Point p1 = {10, 20};
Point p2;
p2 = p1;        // Copy all members: p2.x = 10, p2.y = 20
p2.x = 30;      // Modify member
// p1 == p2;    // Error: equality not supported for structs
```

Variable Declaration

TyC supports two ways to declare variables:

1. **Type inference using `auto`**: The type can be inferred from the initialization expression (if present)
2. **Explicit type declaration**: The type is explicitly declared using type keywords (`int`, `float`, `string`, or struct type names)

Variable Declaration Forms

With `auto` and initialization (type inference):

```
auto <identifier> = <expression>;
```

With `auto` without initialization:

```
auto <identifier>;
```

Note: When using `auto` without initialization, the variable's type must be determined from subsequent assignments or usages. However, if the variable is used before being assigned, an error occurs.

With explicit type and initialization:

```
<type> <identifier> = <expression>;
```

With explicit type without initialization:

```
<type> <identifier>;
```

Where `<type>` is one of: `int`, `float`, `string`, or a struct type name

Examples

```
// With auto and initialization
auto x = 10;           // x is int (inferred)
auto y = 3.14;         // y is float (inferred)
auto msg = "hello";    // msg is string (inferred)
auto sum = x + y;      // sum is float (inferred)

// With auto without initialization (type inferred from first usage)
auto a;
a = 10;                // a is int (inferred from first usage - assignment)
auto b;
b = 3.14;              // b is float (inferred from first usage - assignment)
auto c;
c = readInt();         // c is int (inferred from first usage - function return
type)

// With explicit type and initialization
int x = 10;
float d = 3.14;
string s = "hello";
int result = x + 5;

// With explicit type without initialization
int e;
float f;
string t;
e = 10;                // e is already int
f = 3.14;              // f is already float
```

Important Rules:

- When using `auto` with initialization, the type is inferred from the initialization expression
- When using `auto` without initialization, the type is inferred from the first usage of the variable (assignment, expression, function argument, return value, etc.)
- When using explicit type, initialization is optional
- If a variable is used in a context where its type cannot be determined, a semantic error occurs

---

Expressions

Expressions are constructs which are made up of operators and operands. They calculate on their operands and return new data. In TyC, there exist two types of operations, unary and binary. Unary operations work with one operand and binary operations work with two operands.

Arithmetic Expression

Arithmetic expressions use the following operators:

Operator	Operation	Operand Type	Result Type

Operator	Operation	Operand Type	Result Type
`+`	Prefix unary sign identity	int or float	same as operand
`-`	Prefix unary sign negation	int or float	same as operand
`+`	Infix binary addition	int or float	int if both int, else float
`-`	Infix binary subtraction	int or float	int if both int, else float
`*`	Infix binary multiplication	int or float	int if both int, else float
`/`	Infix binary division	int or float	int if both int, else float
`%`	Infix binary remainder	int	int

The operands of `+`, `-`, `*`, `/` can be of **int** or **float** type. If both operands are int, the result is int. If at least one operand is float, the result is float.

The operands of `%` must be of **int** type only, and the result is always **int**.

Relational Expression

Relational operators perform comparisons on their operands. The operands can be of **int** or **float** type. All relational operations result in an **int** type (0 for false, non-zero for true). Relational operators include:

Operator	Meaning	Operand Type	Result Type
`==`	Equal	int or float	int
`!=`	Not equal	int or float	int
`>`	Greater than	int or float	int
`<`	Less than	int or float	int
`>=`	Greater than or equal	int or float	int
`<=`	Less than or equal	int or float	int

Logical Expression

Logical expressions have logical operators, such as `&&` (AND), `||` (OR), `!` (NOT). The operands of these operators must be of **int** type (0 is false, non-zero is true), and their result type is also **int** (0 for false, non-zero for true).

Increment and Decrement Expression

TyC supports prefix and postfix increment/decrement operators. These operators apply to **int** type only:

Operator	Operation	Operand Type	Result Type
`++`	Prefix increment	int	int
`--`	Prefix decrement	int	int

Operator	Operation	Operand Type	Result Type
`++`	Postfix increment	int	int
`--`	Postfix decrement	int	int

The operand must be of **int** type. Float values cannot be used with increment/decrement operators.

Function Call Expression

A **function call** is an expression that invokes a function. It has the form:

```
<identifier>(<argument_list>)
```

where `<argument_list>` is a comma-separated list of expressions (or empty). The type of a function call expression is the return type of the function being called.

Assignment Expression

An **assignment expression** assigns a value to a variable and can be used as an expression. It has the form:

```
<identifier> = <expression>
```

or

```
<member_access> = <expression>
```

Assignment expressions are right-associative, allowing chained assignments such as `x = y = z = 10;`, which is parsed as `x = (y = (z = 10));`. Assignment expressions can be used in expression contexts, for example: `int y = (x = 5) + 7;`.

Operator Precedence and Associativity

The order of precedence for operators is listed from highest to lowest:

Operator	Associativity
`.` (member access)	left
`++`, `--` (postfix)	left
`++`, `--` (prefix)	right
`!`, `-` (unary), `+` (unary)	right
`*`, `/`, `%`	left
`+`, `-` (binary)	left

Operator	Associativity
`<`, `<=`, `>`, `>=`	left
`==`, `!=`	left
`&&`	left
`||`	left
`=`	right

Evaluation Order

TyC requires the left-hand operand of a binary operator must be evaluated first before any part of the right-hand operand is evaluated. Similarly, in a function invocation, the actual parameters must be evaluated from left to right.

Every operand of an operator must be evaluated before any part of the operation itself is performed. The two exceptions are the logical operators `&&` and `||`, which are still evaluated from left to right, but it is guaranteed that evaluation will stop as soon as the truth or falsehood is known. This is known as the short-circuit evaluation.

Statements

A statement, which does not return anything (except return statement), indicates the action a program performs. There are many kinds of statements, as described as follows. Note that a semicolon (`;`) by itself does not constitute a valid statement; it must be part of a complete statement such as an expression statement, variable declaration, or other statement types.

Variable Declaration Statement

A **variable declaration** declares a variable. The initialization expression is optional:

With type inference (using `auto`):

```
auto <identifier> = <expression>;    // with initialization
auto <identifier>;                    // without initialization
```

With explicit type:

```
<type> <identifier> = <expression>;  // with initialization
<type> <identifier>;                  // without initialization
```

Where `<type>` is one of: `int`, `float`, `string`, or a struct type name

Type Inference Rules:

- When using `auto` with initialization: the type is inferred from the initialization expression
- When using `auto` without initialization: the type is inferred from the first usage of the variable (assignment, expression, function argument, etc.)
- When using explicit type with initialization: the type of the initialization expression must match the declared type
- When using explicit type without initialization: the variable has the explicitly declared type

For example:

```
// With auto and initialization
auto x = 10;           // x is int (inferred)
auto y = 3.14;         // y is float (inferred)
auto msg = "hello";    // msg is string (inferred)
auto sum = x + y;      // sum is float (inferred)

// With auto without initialization
auto a;
a = 10;                // a is int (inferred from first usage - assignment)

// With explicit type and initialization
int b = 10;
float c = 3.14;
string s = "hello";

// With explicit type without initialization
int d;
float e;
string t;
```

Block Statement

A block statement begins with the left brace `{` and ends with the right brace `}`. Between the two braces, there may be a list of variable declarations and statements.

For example:

```
{
    auto x = 10;
    auto y = 20;
    auto sum = x + y;
    printInt(sum);
}
```

If Statement

The **if statement** conditionally executes one of two statements based on the value of an expression. The form of an if statement is:

```
if (<expression>) <statement>
```

or

```
if (<expression>) <statement> else <statement>
```

where `<expression>` evaluates to an **int** value (0 is false, non-zero is true). If the expression results in non-zero then the first `<statement>` is executed. If `<expression>` evaluates to 0 and an else clause is specified then the `<statement>` following `else` is executed. If no else clause exists and expression is 0 then the if statement is passed over.

When nested if statements are used, an `else` clause is always associated with the nearest (innermost) `if` statement that does not already have an `else` clause. For example, in `if (x) if (y) a; else b;`, the `else` is associated with `if (y)`, not `if (x)`.

The following is an example of an if statement:

```
if (flag) {
    printInt(1);
} else {
    printInt(0);
}
```

While Statement

The **while statement** allows repetitive execution of one or more statements. A while statement executes a loop while a condition is true. While statements take the following form:

```
while (<expression>) <statement>
```

The `<expression>` must evaluate to an **int** value (0 is false, non-zero is true). The `<statement>` is executed repeatedly as long as the expression evaluates to non-zero.

For example:

```
auto i = 0;
while (i < 10) {
    printInt(i);
    ++i;
}
```

For Statement

The **for statement** allows repetitive execution of one or more statements. For statements take the following form:

```
for (<init>; <condition>; <update>) <statement>
```

Where:

- `<init>` is a variable declaration or assignment (optional)
- `<condition>` is an expression that evaluates to int (optional, if omitted, treated as always true)
- `<update>` is an assignment, increment, or decrement (optional)

For example:

```
for (auto i = 0; i < 10; ++i) {
    printInt(i);
}
```

Switch Statement

The **switch statement** allows selection among multiple statements based on the value of an expression. Switch statements take the following form:

```
switch (<expression>) {
    case <case_expression>:
        <statement_list>
    case <case_expression>:
        <statement_list>
    ...
    default:
        <statement_list>
}
```

Where:

- `<expression>` must evaluate to an **int** value
- `<case_expression>` is an expression that evaluates to an **int** value. The case expression can be:
  - An integer literal: `case 1:`, `case 42:`
  - An integer literal with unary operators: `case +1:`, `case -5:`
  - A parenthesized expression: `case (1):`, `case (2+3):`
  - A constant expression (expressions involving only integer literals and operators): `case 1+2:`, `case (3*4):`
  - Any other expression that evaluates to an int value at compile time
- Each `case` label must be followed by a colon (`:`)
- The `default` clause is **optional** and can appear anywhere within the switch statement. **At most one `default` clause is allowed** - if multiple `default` clauses are present, it is a compile-time error.

- The switch body can be empty (no case statements and no default clause): `switch (x) { }`
- `<statement_list>` within each case or default can be empty or contain one or more statements
- Like C, **TyC switch statements have fall-through behavior** - execution continues to the next case unless a `break` statement is used

Important: In TyC, switch statements follow C-style fall-through behavior. Execution will fall through to subsequent cases unless explicitly terminated with a `break` statement. You can use multiple case labels for the same code block to handle multiple values.

For example:

```
auto day = 2;
switch (day) {
    case 1:
        printInt(1);
        break;
    case 2:
    case 3:
        printInt(2);
        break;
    default:
        printInt(0);
}

// Empty switch body is valid
switch (x) { }

// Case with constant expressions
switch (x) {
    case 1+2:        // Valid: constant expression
        printInt(3);
        break;
    case (4):        // Valid: parenthesized expression
        printInt(4);
        break;
    case +5:         // Valid: unary plus
        printInt(5);
        break;
    case -6:         // Valid: unary minus
        printInt(6);
        break;
}
```

Break Statement

Using **break statement** we can leave a loop or switch statement even if the condition for its end is not fulfilled. It can be used to end an infinite loop, or to force it to end before its natural end. It must reside in a loop (i.e., in a while or for loop) or in a switch statement. Otherwise, an error will be generated (This will be discussed in Semantic Analysis phase). A break statement is written as follows.
`break;`

In a `switch` statement, `break` terminates the switch statement and transfers control to the statement following the switch. Without `break`, execution will fall through to the next case label (fall-through behavior). `break` is typically used at the end of each case block to prevent fall-through to subsequent cases.

Continue Statement

The **continue statement** causes the program to skip the rest of the loop in the current iteration as if the end of the statement block had been reached, causing it to jump to the start of the following iteration. It must reside in a loop (i.e., in a while or for loop). Otherwise, an error will be generated (This will be discussed in Semantic Analysis phase). A continue statement is written as follows.
`continue;`

Return Statement

A **return statement** aims at transferring control to the caller of the function that contains it. It must be of the form:
`return <expression>;`

or for void functions:

`return;`

The type of the expression must match the return type of the function.

Expression Statement

An **expression statement** is an expression followed by a semicolon. Expression statements are used for their side effects (such as function calls, assignments, or increment/decrement operations).

An assignment expression can be used as an expression statement. When used as a statement, the assignment performs the side effect of updating the variable's value. The type of the value returned by the expression must match the type of the variable being assigned.

For example:

```
printInt(x);      // function call expression statement
x = 5;            // assignment expression statement
x = x + 1;        // assignment expression statement
x++;              // increment expression statement
x + y;            // valid but does nothing useful
```

Type Inference

Complete Type Inference System

TyC uses complete type inference where types are inferred throughout the program. The compiler infers types from multiple sources as described below.

Type Inference Rules

Rule 1: Literal Type Inference

Literals have inherent types:

- **Integer literals** (`123`, `-45`, `0`) → type `int`
- **Float literals** (`9.0`, `12e8`, `1.`, `0.33E-3`, `-3.14`) → type `float`
- **String literals** (`"hello"`, `"world"`) → type `string`

Rule 2: Variable Declaration Type Inference

2.1 Variable Declaration with `auto` and Initialization:

- The type is inferred from the initialization expression
- The variable's type is the type of the initialization expression

```
auto x = 10;            // x: int (from integer literal)
auto y = 3.14;          // y: float (from float literal)
auto msg = "hello";     // msg: string (from string literal)
auto z = x + y;         // z: float (from expression result type)
```

2.2 Variable Declaration with `auto` without Initialization:

- The type is inferred from the first usage of the variable
- The variable's type is determined by the context of its first usage:
  - If first usage is an assignment: type is the type of the right-hand side expression
  - If first usage is in an expression: type is determined by the expression's type requirements
  - If first usage is as a function argument: type is determined by the function parameter type
  - If first usage is as a return value: type is determined by the function return type
- If the variable is used in a context where its type cannot be determined, a semantic error occurs

```
auto a;                 // type unknown initially
a = 10;                 // a: int (inferred from assignment - first usage)
auto b;
b = 3.14;               // b: float (inferred from assignment - first usage)
auto c;
c = a + b;              // c: float (inferred from expression - first usage)

auto x;
x = readInt();          // x: int (inferred from function return type - first
usage)

auto y;
printInt(y);            // y: int (inferred from function parameter type - first
usage)
auto z;
z = readInt();          // z: int (inferred from function return type - first
usage)
printInt(z);            // Now z is int, so printInt can be used
```

2.3 Variable Declaration with Explicit Type and Initialization:

- The variable's type is the explicitly declared type
- The type of the initialization expression must match the declared type (type checking required)

```
int x = 10;             // x: int (explicit)
float y = 3.14;         // y: float (explicit)
string s = "hello";     // s: string (explicit)
int z = x + 5;          // z: int (explicit, expression must evaluate to int)
```

2.4 Variable Declaration with Explicit Type without Initialization:

- The variable's type is the explicitly declared type
- The variable has an undefined value until assigned

```
int a;                  // a: int (explicit)
float b;                // b: float (explicit)
string c;               // c: string (explicit)
a = 10;                 // assignment to int variable
```

Rule 3: Expression Type Inference

The type of an expression is inferred from its components:

3.1 Primary Expressions:

- **Identifier**: Type is the declared type of the identifier
  - If identifier is a variable: type is the variable's type
  - If identifier is a function: type is the function's return type (function call)
- **Literal**: Type is determined by Rule 1
- **Parenthesized Expression**: `(expr)` → type is the type of `expr`
- **Member Access**: `expr.memberName` → type is the type of the struct member
  - `expr` must be of a struct type
  - `memberName` must be a member of that struct type

3.2 Unary Expression Type Inference:

Expression	Operand Type	Result Type
`+expr`	`int` or `float`	same as operand
`-expr`	`int` or `float`	same as operand
`!expr`	`int`	`int`

Expression	Operand Type	Result Type
`++expr`	`int`	`int`
`--expr`	`int`	`int`

3.3 Postfix Expression Type Inference:

Expression	Operand Type	Result Type
`expr++`	`int`	`int`
`expr--`	`int`	`int`
`expr(args)`	function call	function's return type

3.4 Binary Expression Type Inference:

Arithmetic Operators:

Operator	Left Operand	Right Operand	Result Type
`+`, `-`, `*`, `/`	`int`	`int`	`int`
`+`, `-`, `*`, `/`	`int`	`float`	`float`
`+`, `-`, `*`, `/`	`float`	`int`	`float`
`+`, `-`, `*`, `/`	`float`	`float`	`float`
`%`	`int`	`int`	`int`

Relational Operators:

Operator	Left Operand	Right Operand	Result Type
`==`, `!=`, `<`, `<=`, `>`, `>=`	`int` or `float`	`int` or `float`	`int`

Logical Operators:

Operator	Left Operand	Right Operand	Result Type
`&&`, `||`	`int`	`int`	`int`
`!`	`int`	N/A (unary)	`int`

3.5 Function Call Expression Type Inference:

- **Function Call**: `identifier(args)` → type is the return type of the function being called
- The function's return type can be explicitly declared or inferred from the function's return statements

3.6 Assignment Expression Type Inference:

- **Assignment**: `identifier = expr` → type is the type of `identifier` (left-hand side)
- The type of `expr` must match the type of `identifier`

Rule 4: Type Compatibility and Checking

4.1 Assignment Compatibility:

- Left-hand side type must match right-hand side type
- For explicit type declarations: initialization expression type must match declared type

4.2 Operator Type Compatibility:

- Each operator applies to specific operand types as described in Rule 3
- Operand types must be compatible with the operator's requirements

4.3 Function Call Type Compatibility:

- Argument types must match parameter types
- Number of arguments must match number of parameters

Rule 5: Function Return Type Declaration

- Function return types can be explicitly declared or omitted (inferred)
- When the return type is omitted, it is inferred from the return statements in the function:
  - The return type is inferred from the **first return statement** that returns a value
  - If there are no return statements or only `return;` statements, the return type is inferred as `void`
  - All subsequent return statements must return a value of the inferred return type - if a return statement returns a value of a different type, it is a compile-time error (TypeMismatchInStatement)
- All `return` statements in a function must return a value of the inferred/declared return type (or no value for `void`)

Strict Operator Typing

Each operator applies to specific types:

- **Arithmetic operators** (`+`, `-`, `*`, `/`): operate on `int` or `float` (result is `int` if both operands are int, otherwise `float`)
- **Modulus operator** (`%`): operates on `int` only (result is `int`)
- **Relational operators** (`==`, `!=`, `<`, `<=`, `>`, `>=`): operate on `int` or `float` (result is always `int`)
- **Logical operators** (`&&`, `||`, `!`): operate on `int` only (result is `int`, 0 is false, non-zero is true)
- **Increment/decrement** (`++`, `--`): operate on `int` only (result is `int`)

Restrictions:

- You cannot add two strings: `"hello" + "world"` is a type error
- You cannot compare strings: `"a" < "b"` is a type error
- You cannot use modulus on float values: `3.14 % 2` is a type error
- You cannot use logical operators on float values: `3.14 && 2.5` is a type error
- You cannot use increment/decrement on float values: `++x` where x is float is a type error
- You cannot use arithmetic/relational/logical operators on string values

Type Inference Examples

```
// Rule 2.1: auto with initialization
auto x = 10;              // x: int (from integer literal)
auto y = 20;              // y: int (from integer literal)
auto z = x + y;           // z: int (both operands are int)
auto f = 3.14;            // f: float (from float literal)
auto g = 2.5;             // g: float (from float literal)
auto h = f + g;           // h: float (at least one operand is float)
auto mixed = x + f;       // mixed: float (one operand is float)
auto flag = x < y;        // flag: int (relational operator returns int)
auto result = flag && 1;  // result: int (logical operator returns int)
auto msg = "hello";       // msg: string (from string literal)

// Rule 2.2: auto without initialization
auto a;
a = 10;                   // a: int (inferred from first usage - assignment)
auto b;
b = 3.14;                 // b: float (inferred from first usage - assignment)
auto c;
c = a + b;                // c: float (inferred from first usage - expression)
auto d;
d = readInt();            // d: int (inferred from first usage - function return
type)

// Rule 2.3: explicit type with initialization
int e = 10;
float f = 3.14;
string s = "hello";
int sum1 = e + 5;         // sum1: int (explicit, expression evaluates to int)

// Rule 2.4: explicit type without initialization
int i;
float j;
string t;
i = 10;                   // assignment to int variable
j = 3.14;                 // assignment to float variable

// Rule 3: Expression type inference examples
auto expr1 = 10 + 20;           // expr1: int (both operands int)
auto expr2 = 10 + 3.14;         // expr2: float (one operand float)
auto expr3 = 3.14 + 2.5;        // expr3: float (both operands float)
auto expr4 = 10 % 3;            // expr4: int (modulus returns int)
auto expr5 = 10 < 20;           // expr5: int (relational returns int)
auto expr6 = 10 && 1;           // expr6: int (logical returns int)
auto expr7 = ++x;               // expr7: int (increment returns int)

// Function declarations (Rule 5)
int add(int a, int b) {
    return a + b;         // returns int
}

float multiply(float a, float b) {
    return a * b;         // returns float
}
```

```
// Function calls (Rule 3.5)
auto sum2 = add(5, 3);              // sum2: int (function returns int)
int sum3 = add(5, 3);               // sum3: int (explicit type)
auto product1 = multiply(2.5, 3.0); // product1: float (function returns float)
float product2 = multiply(2.5, 3.0); // product2: float (explicit type)
```

Scope Rules

There are 2 levels of scope: global and local.

Global Scope

All function names and struct names have global scope. A function name or struct name is visible everywhere in the program. Functions can be invoked from any function, and struct types can be used throughout the program.

Local Scope

All variables declared in a function (including parameters) have local scope. They are visible from the place they are declared to the end of the enclosing block or function. Variables declared in nested blocks shadow variables with the same name in outer scopes.

Input and Output

To perform input and output operations, TyC provides the following built-in functions:

Function Prototype	Semantic
`int readInt();`	Read an integer number from keyboard
`float readFloat();`	Read a floating-point number from keyboard
`string readString();`	Read a string from keyboard
`void printInt(int value);`	Write an integer number to the screen
`void printFloat(float value);`	Write a floating-point number to the screen
`void printString(string value);`	Write a string to the screen

Example Programs

Example 1: Hello World

```
void main() {
    printString("Hello, World!");
}
```

Example 2: Simple Calculator

```
int add(int x, int y) {
    return x + y;
}

int multiply(int x, int y) {
    return x * y;
}

void main() {
    auto a = readInt();
    auto b = readInt();

    auto sum = add(a, b);
    auto product = multiply(a, b);

    printInt(sum);
    printInt(product);
}
```

Example 3: Loop with Conditions

```
void main() {
    auto n = readInt();
    auto i = 0;

    while (i < n) {
        printInt(i);
        ++i;
    }

    for (auto j = 0; j < n; ++j) {
        if (j % 2 == 0) {
            printInt(j);
        }
    }
}
```

Example 4: Factorial Function

```
int factorial(int n) {
    if (n <= 1) {
        return 1;
    } else {
        return n * factorial(n - 1);
    }
}
```

```
void main() {
    auto num = readInt();
    auto result = factorial(num);
    printInt(result);
}
```

Example 5: Variable Declaration with Optional Initialization

```
void main() {
    // With auto and initialization
    auto x = readInt();
    auto y = readFloat();
    auto name = readString();

    // With auto without initialization
    auto sum;
    sum = x + y;              // sum: float (inferred from first usage -
assignment)

    // With explicit type and initialization
    int count = 0;
    float total = 0.0;
    string greeting = "Hello, ";

    // With explicit type without initialization
    int i;
    float f;
    i = readInt();            // assignment to int
    f = readFloat();          // assignment to float

    printFloat(sum);
    printString(greeting);
    printString(name);

    // Note: String concatenation is NOT supported
    // This is because + operator applies to int or float, not string
}
```

Example 6: Struct Usage

```
struct Point {
    int x;
    int y;
};

struct Person {
    string name;
    int age;
```

```
    float height;
};

void main() {
    // Struct variable declaration without initialization
    Point p1;
    p1.x = 10;
    p1.y = 20;

    // Struct variable declaration with initialization
    Point p2 = {30, 40};

    // Access and modify struct members
    printInt(p2.x);
    printInt(p2.y);

    // Struct assignment
    p1 = p2;  // Copy all members

    // Person struct usage
    Person person1 = {"John", 25, 1.75};
    printString(person1.name);
    printInt(person1.age);
    printFloat(person1.height);

    // Modify struct members
    person1.age = 26;
    person1.height = 1.76;

    // Using struct with auto
    auto p3 = p2;  // p3: Point (inferred from assignment)
    printInt(p3.x);
}
```

Grammar Summary

A TyC program consists of a sequence of struct declarations and function declarations.

Struct Declarations:

- Each struct declaration defines a new composite type with named members
- Struct members must have explicit types (`int`, `float`, `string`, or another struct type)
- Struct members cannot use `auto` for type inference
- **Nested struct definitions are not supported:** Struct declarations cannot be nested (you cannot declare a struct inside another struct declaration)
- **Struct members can be struct types:** However, struct members can be of other struct types (using previously declared struct types)

Function Declarations:

- Each function has:

    - An optional return type (or can be inferred)
    - An identifier (function name)
    - A parameter list (optional)
    - A block containing statements
- Function names must be unique (no function overloading)

The main structural elements include:

- **Structs**: Composite types with named members of explicit types
- **Functions**: Declarations with return types, parameters, and bodies
- **Statements**: Variable declarations, control flow (if, while, for, switch-case), break, continue, return, expression statements (including assignments), and blocks
- **Expressions**: Primary expressions (identifiers, literals, parenthesized, member access), unary operations, binary operations following operator precedence, function calls, and postfix operations (increment/decrement)
- **Types**: `int`, `float`, `string`, `void`, struct types, and type inference using `auto`
- **Variable Declaration**: Can use `auto` for type inference or explicit types (`int`, `float`, `string`, or struct type names)
- **Literals**: Integer, floating-point, and string literals

Operator Precedence (as specified in the Expressions section, highest to lowest):

1. Member access (`.`)
2. Postfix operators (`++`, `--`)
3. Prefix/unary operators (`!`, `-`, `+`, `++`, `--`)
4. Multiplicative (`*`, `/`, `%`)
5. Additive (`+`, `-`)
6. Relational (`<`, `<=`, `>`, `>=`)
7. Equality (`==`, `!=`)
8. Logical AND (`&&`)
9. Logical OR (`||`)
10. Assignment (`=`)

Note: The complete grammar rules must be defined in the ANTLR4 grammar file (`TyC.g4`) by the student. This specification provides the requirements and examples, but the detailed grammar implementation is part of the assignment.

End of Specification